CMSC132: Nested Classes and Initialization Blocks

Chapter 8

By Naren Ayinala

May 2026

Contents

1 What Is a Nested Class? **2**

2 Static Nested Classes **2**

3 Member Inner Classes **2**

4 Local Classes Inside Methods **3**

5 Anonymous Classes **3**

6 Anonymous Classes vs Lambda Expressions **4**

7 Static Initialization Blocks **4**

8 Instance Initialization Blocks **5**

9 Mixing Static Fields, Instance Blocks, and Constructors **5**

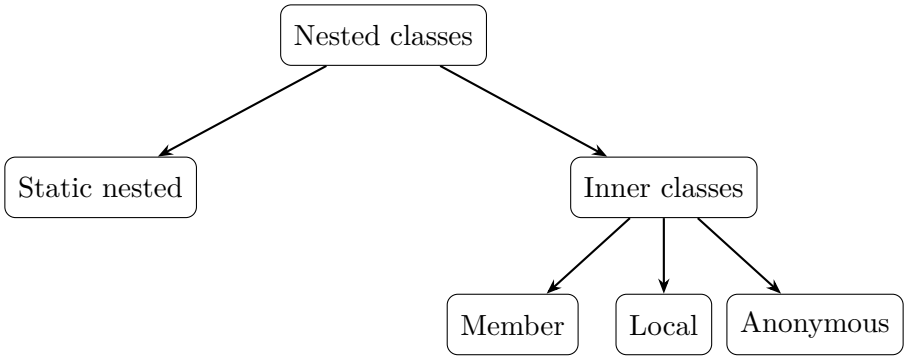
10 Initialization Order with Inheritance **5**

11 Subtleties and Common Mistakes **6**

12 Practice with Answers **6**

1 What Is a Nested Class?

A nested class is simply a class declared inside another class. Java divides nested classes into **static nested classes** and **inner classes**. Inner classes include member inner classes, local classes, and anonymous classes. The distinction matters because some nested classes are tied to an instance of the outer class and others are not.



2 Static Nested Classes

A static nested class belongs to the enclosing *class*, not to a particular outer object. The keyword `static` here does *not* mean all of the nested class's methods must be static. A static nested class may have instance fields, constructors, and instance methods.

```
class Outer {
    private static int count = 10;
    private int id = 42;

    static class Helper {
        void show() {
            System.out.println(count); // static outer field
            // System.out.println(id); // no outer instance
            Outer obj = new Outer();
            System.out.println(obj.id); // valid
        }
    }
}
Outer.Helper h = new Outer.Helper();
h.show();
```

Nested classes can access private members of their enclosing class (and vice versa), subject to whether an instance is needed. A static nested class can directly access outer static members but must use an `Outer` reference for outer instance members.

3 Member Inner Classes

A non-static member inner class is associated with an instance of the outer class. Therefore, creating it requires an outer object. The inner object can access the outer object's fields, including private fields, without a separate reference.

```
class School {
```

```
    private String name = "UMD";
    class Student {
        void introduce() {
            System.out.println(name); // outer instance field
            System.out.println(School.this.name);
        }
    }
}
School school = new School();
School.Student student = school.new Student();
student.introduce();
```

Exam trap: `new School.Student()` is not the correct syntax for a non-static member inner class. We need an enclosing instance, as in `school.new Student()`.

4 Local Classes Inside Methods

A local class is declared inside a block, often inside a method. Its name is visible only within that block. It can access enclosing instance fields and local variables that are **final or effectively final**. An effectively final variable is one whose value is never reassigned after initialization.

```
class Greeter {
    void greet(String person) {
        String prefix = "Hello"; // effectively final
        class Message {
            void print() {
                System.out.println(prefix + ", " + person);
            }
        }
        new Message().print();
        // prefix = "Hi"; // would break effective finality
    }
}
```

The local class can still modify fields of the outer object; the effectively-final restriction concerns *captured local variables*. It also does not mean that an object referenced by a captured variable must be immutable.

5 Anonymous Classes

An anonymous class defines and instantiates a class in one expression without giving it a declared class name. It is often used to implement a small interface or customize a class for a single use.

```
Runnable task = new Runnable() {
    @Override
    public void run() {
        System.out.println("Working...");
    }
};
task.run();
```

An anonymous class can declare additional methods, but if the reference type is `Runnable`, those extra methods are not callable through that reference unless the reference type exposes them. An anonymous class has no explicit constructor declaration, but it can use instance initialization blocks.

```
abstract class Animal {
    abstract void speak();
}
Animal a = new Animal() {
    private int times = 0;
    { System.out.println("anonymous object initialized"); }
    @Override void speak() {
        times++;
        System.out.println("Meow " + times);
    }
};
a.speak();
```

6 Anonymous Classes vs Lambda Expressions

For a **functional interface** (an interface with one abstract method), a lambda is often shorter than an anonymous class. A lambda does not create a named class body with its own `this`; inside a lambda, `this` refers to the enclosing instance. Inside an anonymous class, `this` refers to the anonymous object.

```
Runnable anonymous = new Runnable() {
    @Override public void run() { System.out.println("A"); }
};
Runnable lambda = () -> System.out.println("B");

Comparator<String> byLength = (a, b) ->
    Integer.compare(a.length(), b.length());
```

7 Static Initialization Blocks

A **static initialization block** executes when the class is initialized, generally once per class loader, not once per object. It can initialize static state and access static members directly. Static fields and static blocks execute in textual order within their class.

```
class Counter {
    static int count = 1;
    static {
        count += 4;
        System.out.println("static: " + count);
    }
    Counter() { System.out.println("constructor"); }
}
new Counter();
new Counter();
// static: 5
// constructor
// constructor
```

Important correction: The class is initialized on its first active use under normal JVM rules, not necessarily merely when its bytecode is loaded. Static initialization cannot use `this` or `super` because no current instance is involved.

8 Instance Initialization Blocks

An **instance initialization block** executes each time an object is constructed, after the superclass constructor returns and before that class's constructor body. Instance field initializers and instance blocks execute in source order. They are useful for setup shared by multiple constructors, although helper methods or constructor chaining are often clearer.

```
class Widget {
    int x = 2;
    { x += 3; System.out.println("block: " + x); }
    Widget() { System.out.println("constructor: " + x); }
}
new Widget();
// block: 5
// constructor: 5
```

9 Mixing Static Fields, Instance Blocks, and Constructors

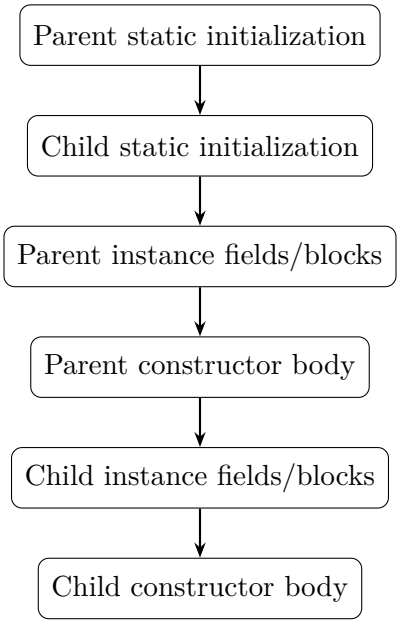
The easiest way to solve output-tracing questions is to separate **class initialization** from **object construction**. Class initialization happens once, whereas instance initialization and constructors happen for each new object.

```
class Demo {
    static int a = announce("static field");
    static { announce("static block"); }
    int b = announce("instance field");
    { announce("instance block"); }
    Demo() { announce("constructor"); }
    static int announce(String s) {
        System.out.println(s);
        return 1;
    }
}
new Demo();
new Demo();
```

The output is `static field`, `static block`, then `instance field`, `instance block`, `constructor`; the last three lines repeat for the second object.

10 Initialization Order with Inheritance

For a first active use of a subclass, the superclass is initialized before the subclass. When constructing an object, the superclass's instance initialization and constructor execute before the subclass's instance initialization and constructor body.




```
class Parent {
    static { System.out.println("P static"); }
    { System.out.println("P block"); }
    Parent() { System.out.println("P ctor"); }
}
class Child extends Parent {
    static { System.out.println("C static"); }
    { System.out.println("C block"); }
    Child() { System.out.println("C ctor"); }
}
new Child();
```

The output is `P static`, `C static`, `P block`, `P ctor`, `C block`, `C ctor`. The static lines are omitted on subsequent constructions after both classes have been initialized.

11 Subtleties and Common Mistakes

- A static nested class does not have an implicit outer object; a member inner class does.
- Inner classes may declare static members in modern Java versions; older course materials often teach the pre-Java-16 restrictions. Follow the Java version required by your course.
- A captured local variable must be effectively final, but the object it refers to may still be mutable.
- A constructor's explicit `super(...)` or `this(...)` invocation follows Java constructor-invocation rules; it cannot be placed inside an instance initialization block.
- A static block runs once per class initialization, not once per instance.

12 Practice with Answers

Q1. How do you construct a member inner class?
Answer: Given `Outer o = new Outer();`, use `Outer.Inner i = o.new Inner();`.

Q2. Can a local class capture a variable that is reassigned later?
Answer: No, because it would not be effectively final.

Q3. How many times does a static block run if we create 100 objects after the class is initialized?
Answer: Zero additional times; it already ran during class initialization.

Q4. Which executes first when creating a child object: parent instance block or child instance block?
Answer: The parent instance block.